

Ami Mail

A user's manual



For mail, the desktop program, and mailc, the same program for a terminal

Petit-Ami, September 2026

1. What this program is

Ami Mail is a mail reader that keeps your mail here rather than on somebody's server. It reads mail from IMAP servers into files on this machine, and everything it shows you it finds by reading those files again.

The rule it is built to, and the one that explains most of what follows: **the server folders are watched, not managed.** They are read and mirrored here. Nothing is created, moved, renamed or flagged on any server. The folders you make are local, they live here, and the servers are never told about them. The one change to a server this program will ever make is taking mail off it, and that is not written yet.

Why keep mail locally

Three reasons, and they build on each other. You get a copy that is yours, in a format any mail program can read. You stop paying for storage you could keep here. And once the mail is a file on your disk, anything can be applied to it: your own rules, your own tools, without asking anybody's permission.

Two programs, one reader

`mail` is the program for the desktop and `mailc` the same program for a terminal. Everything in this manual is true of both; where the terminal differs, it is said.

2. Running it

Either program takes these on the command line:

option	what it does
<code>--fetch,</code> <code>-f</code>	Fetch as soon as the window is up, instead of waiting for Get Mail or the timer.
<code>--diag, -d</code>	Say on standard error what is being done and how long it takes, for finding out why something is slow. The password is held back.
<code>--limit=N</code>	How many messages to take from each folder when an account has not said; 200 to begin with. Config/Servers sets it for each account.

The store is `~/ .amimail`, made on the first run. Beside the program are `mail.md`, the help; `mail.cat`, the rules that say what kind of mail a message is; and `mail.bmp`, the banner's picture. They are looked for where the program is, then in `graph_programs` beside it, then in the current directory, so the program runs from the build directory or from anywhere.

The launcher

`mail.desktop`, beside the program, is a launcher entry for the desktop, with `mail.png` as its icon. Copy the entry to `~/ .local/share/applications` and the picture to `~/ .local/share/icons/hicolor/256x256/apps/ami-mail.png`, and the program is in the launcher and the task bar, with the cat at its laptop for an icon.

What it needs

Printing wants `lp`, the system's print command. A secure connection presents a client certificate, `client_tls_cert.pem` and `client_tls_key.pem`, looked for beside the program, then in the home directory, then in the current directory.

3. Getting started

Config/Servers asks for an account: where the mail is read from, who to log in as, and how much of each folder to take at a time. Get Mail fetches. That is the whole of it.

For Gmail the password is not your account password. Google requires an application password, which it issues per program to an account that has two step verification turned on, from `myaccount.google.com/apppasswords`. It is sixteen characters; the spaces it is shown with are not part of it.

The mail server is `imap.gmail.com` on port 993. The sending server is `smtp.gmail.com` on port 465, not 587, which begins in the clear and expects STARTTLS, which this cannot do.

How many messages

The form asks how many of each folder to fetch. Start small. A mailbox of forty thousand messages is fetched one message at a time, and the first look at it should not be an afternoon. Fetching again takes only what is new, so a small number costs nothing later.

More than one account

Config/Servers holds several. Next walks through them, Add starts another, Remove takes one away. Each has a name of its own, which is how its folders are labelled in the pane and what its directory in the store is called. Renaming an account renames its directory, so nothing has to be fetched again.

Mail is gathered from every account. Where it goes afterwards is not the accounts' business: the local folders are one set, shared. A folder called Bills is about bills, not about which company carried the letter.

Which account sends

One of them. Config/Servers has a box on each account saying it is the one that sends, and ticking it for an account unticks the rest: a message leaves over one connection with one name on it, so this is a choice among the accounts rather than something each of them carries.

4. The window

Across the top, under the menu: the program's name and the picture that goes with it, with a double line under them to say that what is below is a different thing. The picture is `mail.bmp`, kept beside the program; replace it and the banner follows it. If it is missing the banner is the name alone.

Down the left, the folders, in sections: one for each account's folders, then one for the folders that are yours. They are separate lists rather than one list marked up, because two accounts may each have an INBOX and a local Trash is not the server's Trash. The number beside a folder is how many messages are in it here.

On the right, the messages of the folder picked: who each is from, what kind of mail it is, the subject, as much of the message as fits, and when it arrived. The top one is picked when a folder is shown; a click picks another, and a double click, or Return, opens the pick in a window of its own.

Along the foot, a line saying what is being worked on and how far into it, with a bar beside the words when the work has a known size: which account is being connected to, which folder is being read and how many of its messages have arrived, or which mailbox is being counted. It is there because anything that takes longer than an instant has to say so. A program that goes quiet across a wait cannot be told from one that has fallen over.

Looking by itself

The program looks at the servers every so often while it is open, every fifteen seconds unless the form says otherwise. Set it to zero and it will only fetch when you ask. A look on the timer does not ask the servers what folders they have; folders do not come and go by the quarter minute, and asking costs a connection to each server. Get Mail asks; the timer only fetches.

While it fetches

The fetching is done on a thread of its own, so the window answers while it goes on. Folders can be read, messages opened, the window resized, and the mail keeps arriving behind it. A server that stops answering is given up on after forty five seconds and then left alone for a while, fifteen seconds, then thirty, up to a quarter of an hour, so that a server which has had enough of us is not asked again immediately. Nothing is lost by any of it: every folder remembers the stretch it has taken and every message is known by its digest, so the next look carries on from where the last one stopped.

5. Reading

A message opens in a window of its own with Reply, Reply All, Forward and Print on its menu. The wheel moves one message a notch in the list and three lines in an open message, since text is not read a message at a time. The page keys and the scroll bar move the list.

Mail is not text any more, so there is enough MIME here to read it: encoded subject lines, quoted printable, base64, multipart gone into for the plain text part, and HTML with the tags taken out where plain text is not on offer.

The keyboard

The folders and the list each mark their pick: in green at the pane the keyboard is at, in cyan at the other. The program comes up with the keyboard at the list, on the first message of the first folder, so it can be worked from the keys from the start.

key	what it does
Up, Down	Move the mark of the pane the keyboard is at. At the folders, moving the mark shows the folder.
Return	At the list, opens the message marked.
Left, Right	Take the keyboard to the folders or to the list; each does nothing if it is there already.
Alt F12	Puts the keyboard at the menu bar, on Compose. On the desktop, Alt pressed and let go by itself does the same. Left and Right then walk the bar, Down or Return opens an entry's list, Up and Down walk that, Return chooses.
Escape	Leaves the menu, whether the keys or the mouse opened it. Also closes the search form, the server form, the options box, the help window, a message's menu, and a message being read.
Page Up, Page Down	Move the list a page.
/	In the terminal, opens the search form.

The mouse carries the keyboard with it: a click on a folder or on a message puts the keyboard at that pane.

What kind of mail this is

Every message is given a category, shown in its own column. The rules are in a file, `mail.cat`, not in the program: a category and what it matches, one to a line, first match wins. Change them, add to them or throw them away without a compiler.

The rule that does the most work is the last one. Machine-sent mail says so in its headers, List-Unsubscribe, List-Id, Precedence: bulk, so mail carrying none of those, having matched nothing more particular, was written by a person. That division is the one that matters in a mailbox, and headers get it right nearly always.

6. Threads

Config/Options opens a box of check boxes. What each says is kept with the accounts, so it holds next time.

Threaded mode, which is on to begin with, gathers the list into conversations. A thread is the messages with one subject, once the Re: and Fwd: and any tags in brackets are taken off the front. The thread begins with its first message and the replies follow it, oldest first, each standing in under the message it answers, found by its In-Reply-To header, or under the first message when what it answers is not here. The threads stand newest first, by their newest message. Off, the list is every message on its own, newest first.

7. Searching

Search, on the menu bar, opens a form modelled on the one behind the search box in Gmail: who the message is from, who it is to, what the subject holds, words it has, words it must not have, its size, when it came, and which folder to look in. Fill in what you know and leave the rest blank; a message is found when it fits everything given. Return in any field searches, as the Search button does.

A term matches whole, in any case: From "ali" finds Ali and not AliExpress, and a subject is matched against the whole subject line. With Allow wildcards ticked, which it is to begin with, a * in a term stands for any run of characters and a ? for any one: "ali*" finds both, "*@quora.com" finds every Quora digest, and "*stock*" finds every subject with stock in it. Unticked, the two are characters like any other. From is matched against the sender's name and address, To against each name and address on the To line. The words fields are lists of words set apart by spaces; each is matched against the words of the message and its headers, with wildcards in it if they are on.

Size takes a number, in megabytes, kilobytes or bytes as the box beside it says. The date is a day, written as 2026-09-09 or 9/9/2026, and the period beside it is how far either side of that day to look. The folder box offers every folder or all of them, and the box below asks for messages that carry an attachment.

The sender, To, subject, size and date are answered from the index, which is quick. The words and the attachment are answered from the messages themselves, which are read from the store one by one; the line under the buttons says which folder is being read and how far it has got. A search that is asked again before it is done drops what it was doing and starts over.

What is found is listed under the form, newest first: when it came, who from, which folder it is in, and the subject. A click picks one and a double click, or Return, reads it; the arrows, the page keys and the scroll bar move through the list. A result has the same second-button menu a message in the list has, described next, and it acts in the folder the result was found in.

8. Local folders and the second mouse button

Local folders are yours. Made here, filled here, never mentioned to any server. The second mouse button on a message makes one.

On a message, a menu. Read opens it. The rest move mail into local folders, and only local ones; the server is never touched:

entry	what it takes
Move to local Trash	That message, to a local folder called Trash. Not the server's Trash.
Local folder for a place	Everything in this folder from that sender's domain, less whatever it is a part of. LinkedIn writes from four addresses and they are all linkedin.
Local folder for a name	Everything from that display name. Facebook writes under eight of them, one per friend, and sometimes that is what you want kept apart.
Local folder for this thread	The conversation: everything with this subject, Re: and Fwd: aside, whoever wrote it.
Local folder for to an address	One for each address on the message's To line: everything that went to that address. Mail to a list, or to an old address of yours, is a kind of its own.

The menu says how many messages each would take, because which one is right depends on the mail, and the count is what makes it a choice rather than a guess. Escape, or a click anywhere else, puts the menu away.

A search result has the same menu, and it acts in the folder the result was found in: the counts are of that folder, and the move is out of it. The results are made afresh once the move is done, so what was moved is shown where it went. A result found before a move can no longer be acted on, and the menu says so; searching again puts that right.

9. Writing and sending

Compose opens a window with the three fields a message needs and a space to write in. Send sends it; Cancel throws it away. The fields are edit boxes and the body is not, since there is no widget for more than one line of text: it is kept and drawn here, and typing into it works the way typing works, the arrows, Home and End, Page Up and Page Down, Enter to break a line, Backspace and Delete to mend one. A click on a field takes the keys, and a click on the body gives them back.

Sending happens on the same thread as fetching, so a message written while mail is coming in goes at once rather than after, and the window answers throughout. Everything sent goes into a local folder called Sent, in the same form as everything received, indexed the same way. What is sent is ours as much as what arrives.

Answering

Reply goes to whoever wrote it. Reply All goes to them and to everybody else the message was addressed to, less yourself. Both carry the subject with Re: in front of it, quote what is being answered under a line saying who wrote it and when, and put the caret above the quoting where the writing goes. Both are threaded: the answer says which message it answers, so a mail reader at the other end files it in the conversation.

Forward carries the message under a line saying where it came from, with its own subject marked Fwd:, and is not threaded; it is a new message to somebody who was not in the conversation.

Printing

Print sends the message as the reader shows it, the header lines and the text, to the printer, through `lp`, the system's print command, so it goes wherever `lp` is set to send things. The strip says it has gone, or that `lp` would not take it, which on most systems means there is no printer set up.

10. The store

One directory for each account, one called local for yours:

```
~/.amimail/  
  account  
  google/    INBOX.mbox  All_Mail.mbox  Sent_Mail.mbox ...  
  local/     Bills.mbox   Family.mbox   ...
```

Each mailbox is mbox: the message exactly as it arrived, headers and all, with a line before it saying who it is from and when. Any mail program can read it, and this one can read theirs.

Beside each mailbox are two small files. One remembers how far that folder has been read from the server, so fetching again takes only what is new. The other is the index: a line for every message saying where it is in the mailbox and how long it is, what it is, who it is from and who to, what it is about, when it came, the start of what it says, and its message id and the id of the message it answers, which is what the threads are built from.

The index is what the list is drawn from and what the store is searched by, and it is written as the mail arrives. Without one, showing a folder means reading and parsing every byte of its mailbox, half a minute for one with sixty thousand messages in it, every time it is opened. With one it is a third of a second, once, at startup. Nothing has to be done to make it: a folder that has no index gets one the first time it is read, and a folder whose mailbox has been rewritten underneath its index has it taken again. A mailbox that has simply grown costs only the new messages.

Digests

Every message carries the SHA-256 of what it is, as one of the fields of the index. It is how this program knows a message it already has, whoever sent it and whatever folder it arrives in: mail moved between folders keeps its digest, mail fetched twice has the same one, and the same mail from a second server has the same one. A message already here is not written again, and since the digest sits in the index beside the message it belongs to, knowing that it is here also says which folder it is in and where in the file, which is what checking this store against a server will need.

The account file holds a password and is written so that only its owner can read it.

11. What is not here yet

Deleting from a server. That is the one change to a server this program will make, and it will not be made until the mail here has been checked against what is there, message by message.